Automated Environment Processing for High-Fidelity RL

Subproject proposal

October 2026

Goal. Build an open-source pipeline that converts heterogeneous task collections into standardized, verified, and policy-measured RL environment banks. The system should make it inexpensive to assemble controlled candidate environments across domains before launching large RL runs. It automates environment *processing*, while final experimental selection remains explicit and reviewable. This subproject is one component of our broader environment-utility research program.

Why this is needed. An RL task can be unusable because its sandbox is broken, its verifier accepts a no-op, its reference solution fails, its reward is unstable, or the current policy almost always succeeds or fails. Even valid tasks can form a poor study if candidate mixtures overlap heavily or if their measured coordinates are nearly collinear. Downstream RL is the final test, but it is too expensive to serve as the first quality check.

Primary output. The pipeline takes source datasets and a target policy as input and produces:

1. a versioned Harbor-format task bank with pinned source and image hashes,
2. task-level integrity, resource, rollout, and provenance records,
3. reproducible candidate-environment manifests, and
4. a report describing signal, coverage, cost, reliability, overlap, and exclusions.

Concrete example

Suppose the input contains 10,000 coding tasks from repository repair, shell use, and notebook debugging for an 8B policy.

Task	Observation	Processing decision
A	Oracle passes, no-op fails, and 6 of 16 policy rollouts succeed. Rewards vary and infrastructure is stable.	Admit to the measured bank. Record reward variation, generated tokens, admitted advantage mass, and source metadata.
B	Oracle passes, but all 16 policy rollouts succeed.	Keep in a policy-relative reserve. It is valid, but supplies no group-relative signal for this policy under the current rollout contract.
C	Oracle fails or a no-op receives reward 1.	Quarantine as a verifier defect. Do not count the failure as policy reward 0.
D	The verifier is correct, but 5 of 16 sessions fail because the image cannot start.	Quarantine as infrastructure-invalid and retry only after repair under the same task identity.
E	Valid and mixed, but nearly duplicates a held-out benchmark task.	Exclude from training and preserve the contamination record.

The filtered output is not one opaque "good dataset." It is a task bank with explicit admitted, reserve, repair, and excluded states, together with candidate mixtures. For each candidate environment E, the probe computes

$$\rho(E) = \frac{10^6 \sum_g A_g^{\mathrm{adm}}}{\sum_{g,i} \ell_{gi}}, \qquad H(E) = \frac{1}{n_E} \frac{\left(\sum_g A_g^{\mathrm{adm}}\right)^2}{\sum_g \left(A_g^{\mathrm{adm}}\right)^2}.$$

Here ρ measures admitted group-relative signal per million generated tokens, while H measures how broadly that signal is distributed across prompts. These definitions follow our environment-utility study, *Which Environments Should LLMs Learn From? Predicting RL Utility Across Scale*. Candidate construction should span both coordinates without making them redundant. Low correlation is a panel-design objective, not a task-level pass/fail rule.

Pipeline

1. **Acquire and normalize.** Pin source revisions and licenses, assign stable task identities, deduplicate, and convert tasks through source adapters into Harbor format.
2. **Validate fixtures and verifiers.** Check schema, buildability, resource declarations, deterministic oracle success, no-op failure, and repeated verifier stability. Infrastructure errors remain distinct from scientific reward 0.
3. **Probe the target policy.** Run a frozen rollout contract with a configurable group size and at least two independent measurement views. Record success, mixed-group probability, generated cost, admission, ρ contributions, and failure modes.
4. **Construct candidate environments.** Build mixtures with sufficient unique tasks, bounded pairwise overlap, source and verifier diversity, broad variation in ρ and H, and explicit cost and reliability limits.
5. **Confirm and freeze.** Re-measure finalists with an independent rollout seed. Freeze manifests and hashes only when coordinates are reproducible and integrity gates pass.
6. **Validate with RL.** Train a small, prespecified panel. Test whether pre-RL measurements predict held-out utility curves, rankings, and allocation regret. RL validates the processing system without silently redefining its filters.

Implementation

Begin with a Python reference implementation so that schemas, gates, and measurements can change quickly during the research phase. Once those interfaces stabilize, implement the high-throughput catalog, hashing, deduplication, resumable queues, manifest construction, and reporting CLI in Rust. Keep thin Python adapters for model inference, Harbor, and existing RL runtimes. Rust should improve reliability and CPU-side throughput without requiring the project to rebuild the systems that dominate rollout cost. A proposed interface is:

```
envforge ingest <source> --adapter <name>
envforge validate --oracle --noop --repeat 2
envforge probe --model <path> --group-size 16 --views 2
envforge construct --candidates <count> --tasks-per-env <count>
envforge confirm --seed <seed>
envforge freeze --development <manifest> --heldout <manifest>
```

Every command should be resumable, content-addressed, and idempotent. Outputs should use JSON or Parquet schemas with provenance, hashes, error taxonomies, and exact contracts. Secrets, machine-specific paths, and internal experiment notes must never enter released artifacts.

First four weeks

1. **Week 1: literature and specification.** Study DCLM as a precedent for evaluating data curation under a fixed model, training recipe, and compute budget. Then identify the additional requirements introduced by executable RL tasks, including verifiers, sandboxes, policy-relative measurements, and environment mixtures. Compare those requirements with DAPO, MiMo/Uni-Agent, Harbor, and policy-adaptive environment work. Deliver a design document and schema.
2. **Week 2: CPU pipeline.** Implement two source adapters, stable identities, provenance, deduplication, Harbor validation, oracle/no-op gates, and failure taxonomy.
3. **Week 3: policy probe.** Add resumable rollout measurement for one 7 to 8B open model and generate two-view reliability reports for at least 100 tasks.
4. **Week 4: candidate construction.** Produce a diverse candidate collection and a compact development and held-out panel with coordinate, overlap, source-diversity, and reliability audits. Choose their sizes from the available task bank and compute budget.

Initial success criteria. The first version should support at least two domains or source families and reproduce at least 95% of fixture outcomes. It should keep infrastructure failures separate from reward 0, report independent-view reliability for every promoted coordinate, bound candidate overlap, and construct environments with useful variation in ρ and H. Its outputs should integrate cleanly into our broader research pipeline and remain useful to other researchers studying RL environments.

Non-goals for the first version. The project will not generate new tasks, claim that moderate pass rate alone implies utility, automatically choose the final RL environment, or tune filters after observing claim-bearing outcomes. Target affinity or gradient-sketch features may be evaluated later as nested challengers, but they should not become required axes without independent predictive evidence.

Starting references

- Our environment-utility study: policy-relative signal density, effective coverage, and finite-budget RL utility in *Which Environments Should LLMs Learn From? Predicting RL Utility Across Scale*.
- DataComp-LM: controlled filtering and mixing experiments under fixed training contracts, `https://arxiv.org/abs/2406.11794`.
- DAPO: zero-variance group filtering and dynamic refill, `https://papers.nips.cc/paper_files/paper/2025/file/a4277440d50f1f15d2cb4c14f7e0c0d2-Paper-Conference.pdf`.
- Uni-Agent: scalable, traceable agent rollouts and RL integration, `https://github.com/XiaomiMiMo/uni-agent`.
- Harbor task structure and verifier contract, `https://github.com/harbor-framework/harbor/blob/main/docs/content/docs/tasks/index.mdx`.
- RLVE: policy-adaptive difficulty and environment scaling, `https://arxiv.org/abs/2511.07317`.